

評估二: DDoS 攻擊捕獲與數據分析報告



學生姓名: 黃柏凱 PoKai Huang

學生學號: 26254793

1. 引言：數據分析與封包分析在網絡安全中的角色

在高度互聯的數字環境中，網絡安全已從傳統的邊界防禦模式，轉變為以數據驅動的動態防禦體系。隨著攻擊手法的自動化、分散化與高頻化，單純依賴靜態規則與黑名單機制已不足以識別複雜威脅。因此，數據分析在網絡安全中的角色正由描述性分析 (Descriptive Analytics) 進一步邁向診斷性分析 (Diagnostic Analytics)。

描述性分析回答「發生了什麼」

診斷性分析回答「為什麼會發生」，並透過統計建模、時間分佈分析與流量特徵比對揭示攻擊根因。

本實驗對應課程兩項核心學習目標：

- 使用數據分析識別安全漏洞
- 運用數據分析技術保護系統、應用程序或服務

透過對正常流量與攻擊流量的比較分析，我們將量化 SYN 洪泛攻擊對系統造成的資源耗盡影響，並展示封包層數據如何揭示協議層漏洞。

本實驗整合以下工具形成完整分析流程：

- **Wireshark**: 封包捕獲與協議層解碼
- **hping3**: DDoS 攻擊模擬工具
- **Jupyter Notebook (Python)**: 數據清理、統計建模與視覺化分析

2. 實驗過程：數據捕獲與處理路徑

2.1 環境配置

本實驗於 VMware Workstation 虛擬化環境中進行

- 系統類型: Debian 12.x 64-bit
- 安裝模式: Graphical Install
- 正常流量捕獲介面: any
- 攻擊流量捕獲介面: loopback (lo)

2.2 正常流量捕獲(Baseline 建立)

在部署攻擊之前，我們需建立正常網路行為的基準線。為了捕獲真實的外部網頁瀏覽行為，我在正常流量階段選擇了 any 介面，因為正常的對外通訊封包必須經過外部網卡傳輸，若選擇 loopback 則無法記錄到這些流量。

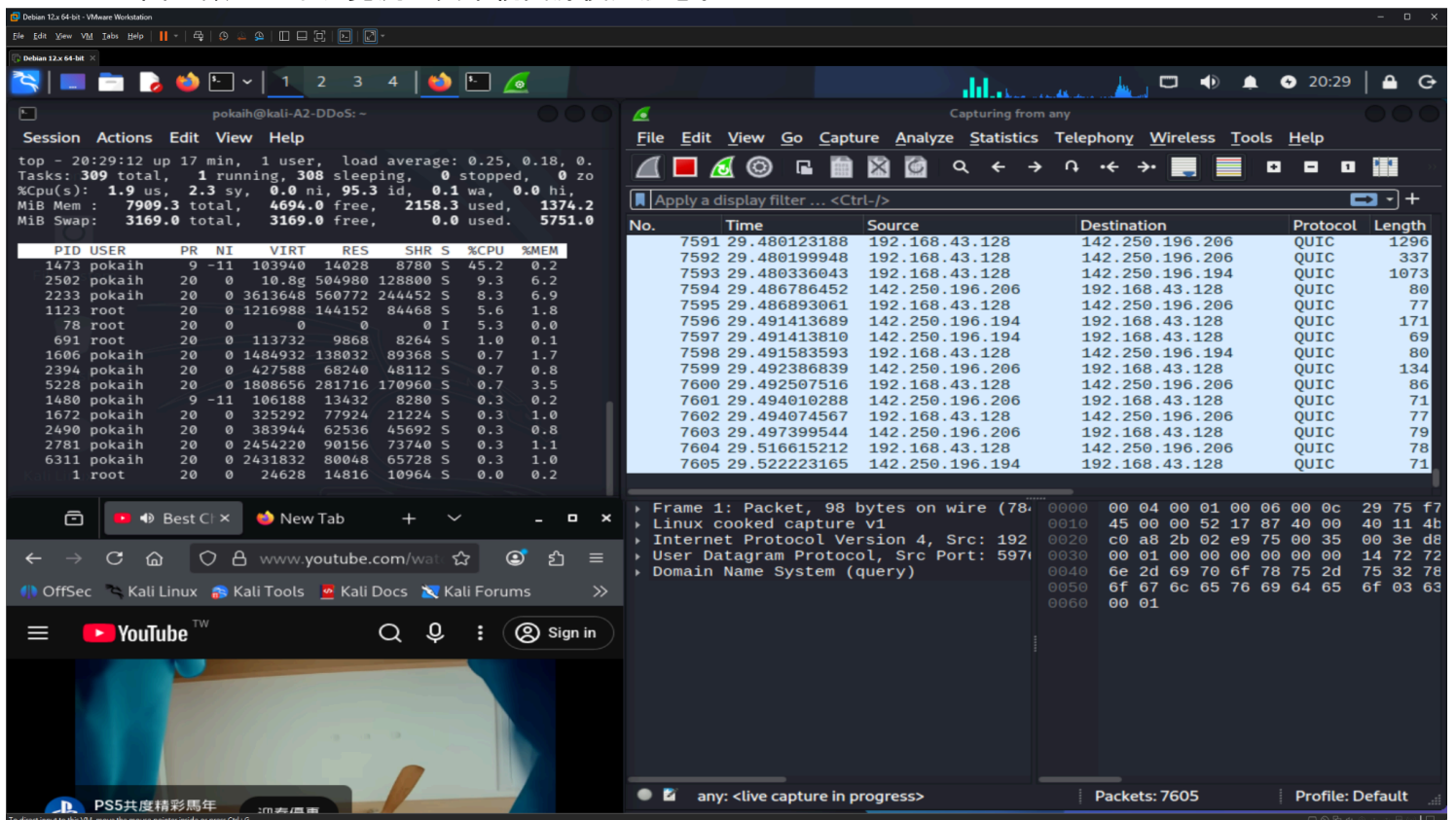
步驟如下：

1. 啟動 Wireshark 並選取 **any** 介面開始捕獲。
2. 使用瀏覽器進行一般網頁瀏覽行為。
3. 停止捕獲並將資料匯出為 CSV 檔案。

在正常流量中，我們可觀察到：

- TCP/TLS 連線建立過程(SYN → SYN-ACK → ACK)
- 或使用 QUIC(HTTP/3)進行資料傳輸
- 封包呈現「短暫突發(bursty)」特性, 而非持續性高頻輸出

圖 1 顯示正常瀏覽流量與系統資源使用狀態。



2.3 DDoS 攻擊部署 (SYN 洪泛)

依照評估二的說明，我們將使用 **hping3** 模擬一個極高壓力的協議層 DDoS 攻擊，為了在安全且受控的虛擬環境下進行測試，避免對外部真實網路造成干擾與法律風險，我在攻擊階段選擇 loopback (lo) 介面，並將目標鎖定為本機 localhost, 127.0.0.1。在終端機執行以下指令：

```
sudo hping3 -S --flood --rand-source -p 80 localhost
```

該指令的參數技術背景：

- **-S (SYN Flag)**: 將 TCP 控制位設定為 SYN, 請求建立新連線。
- **--flood (洪水模式)**: 以網路卡處理極限的速度發送封包, 不等待目標的回應, 引發 CPU 中斷風暴。
- **--rand-source (隨機源地址)**: 偽造海量的隨機源 IP, 使傳統基於黑名單的過濾失效。
- **-p 80**: 鎖定目標的 Web 服務端口, 旨在耗盡其 HTTP 處理能力。

SYN Flood 屬於 **Layer 4** 狀態耗盡型攻擊。當伺服器收到 SYN 封包時，會為該連線分配 TCP 控制塊並進入 SYN_RECV 狀態。若攻擊者不完成三向交握，伺服器將長時間維持半開連線，逐步填滿 backlog queue，最終導致合法連線請求被拒絕。

為避免虛擬機資源耗盡導致系統不穩定，本實驗將攻擊流量捕獲時間控制於 5 秒內。

圖2.執行 hping3 命令及其產生的流量以及資源監視截圖

The screenshot displays a Kali Linux virtual machine environment. The top terminal window shows the output of the `top` command, indicating system load and resource usage. The bottom terminal window shows the output of the `hping3` command, confirming the flood mode and packet loss. The network traffic capture window shows a list of captured packets, with a detailed view of a selected packet (Frame 1) showing the TCP header and source IP address.

System Resource Usage (top output):

```
top - 23:59:09 up 26 min, 1 user, load average: 0.44, 0.52, 0.79
Tasks: 313 total, 3 running, 309 sleeping, 1 stopped, 0 zombi
%Cpu(s): 12.3 us, 12.2 sy, 0.0 ni, 70.5 id, 1.1 wa, 0.0 hi, 3.8
MiB Mem : 7909.3 total, 119.8 free, 7343.7 used, 945.6 bu
MiB Swap: 3169.0 total, 1503.7 free, 1665.3 used, 565.5 av
```

hping3 Command Output:

```
pokaih@kali-A2-DDoS: ~
$ sudo hping3 -S --flood --rand-source -p 80 localhost
HPING localhost (lo 127.0.0.1): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
^C
— localhost hping statistic —
2159903 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
```

Network Traffic Capture (Wireshark):

No.	Time	Source	Destination	Protocol	Length
21149...	4.317664255	249.195.155.130	127.0.0.1	TCP	54
21149...	4.317666413	134.36.250.10	127.0.0.1	TCP	54
21149...	4.317668591	244.195.150.249	127.0.0.1	TCP	54
21149...	4.317670817	232.183.185.120	127.0.0.1	TCP	54
21149...	4.317671445	127.0.0.1	232.183.185.120	TCP	54
21149...	4.317673667	156.78.121.116	127.0.0.1	TCP	54
21149...	4.317675767	192.45.174.156	127.0.0.1	TCP	54
21149...	4.317677904	47.116.40.135	127.0.0.1	TCP	54
21149...	4.317680022	232.95.105.81	127.0.0.1	TCP	54

Packet Details (Frame 1):

- Frame 1: Packet, 54 bytes on wire (43 bytes captured)
- Ethernet II, Src: 00:00:00:00:00:00 (00:00:00:00:00:00), Dst: 00:00:00:00:00:00 (00:00:00:00:00:00)
- Internet Protocol Version 4, Src: 163.172.18.120, Dst: 127.0.0.1
- Transmission Control Protocol, Src Port: 43176, Dst Port: 80

2.4 數據轉換與 Python 加載

捕獲完成後，將 Wireshark 的數據導出為 CSV 格式，以便進行後續結構化分析。由於原始封包數據屬於半結構化資料，若未經整理直接分析，將難以進行統計建模與視覺化。因此，我使用 Pandas 進行資料清理與欄位標準化處理，將 **Time**, **No.**, **Source**, **Destination**, **Protocol**, **Length**, **Info** 等關鍵欄位轉換為適當的數據型別（如 float、int 與 string），並檢查缺失值與異常值。

此步驟的目的在於確保後續分析的準確性與一致性，用於協議比例比較。透過資料預處理，原始封包資料得以轉換為可供診斷性分析使用的結構化資料集，為後續 DDoS 行為識別奠定基礎。

圖3.Normal.csv 資料處理結果截圖

```
===== Normal Traffic (Baseline) =====
Rows (packets): 8,028
Columns: 7

Preview (First 5 rows):

   No.    Time      Source      Destination Protocol  Length  \
0    1  0.000000  192.168.43.128  192.168.43.2      DNS      98
1    2  0.000067  192.168.43.128  192.168.43.2      DNS      98
2    3  0.000068  192.168.43.128  192.168.43.2      DNS      98
3    4  0.007841   192.168.43.2  192.168.43.128      DNS     147
4    5  0.012024   192.168.43.2  192.168.43.128      DNS     159

                                Info
0  Standard query 0x180a HTTPS rr5---sn-ipoxu-u2x...
1  Standard query 0x0733 A rr5---sn-ipoxu-u2xee.g...
2  Standard query 0x0c32 AAAA rr5---sn-ipoxu-u2xe...
3  Standard query response 0x0733 A rr5---sn-ipox...
4  Standard query response 0x0c32 AAAA rr5---sn-i...

Data Types (dtypes):
No.          int64
Time         float64
Source       object
Destination  object
Protocol     object
Length       int64
Info         object
dtype: object

Missing values (total): 0

Empty DataFrame
Columns: [missing_count]
Index: []

Key columns present: ['No.', 'Time', 'Source', 'Destination', 'Protocol', 'Length', 'Info']
Time range: 0.000000 ~ 46.832036 seconds
Length range: 56 ~ 7201 bytes
```

圖4.ddos_syn_flood_lo.csv 資料處理結果截圖

```

===== Attack Traffic (SYN Flood) =====
Rows (packets): 2,320,270
Columns: 7

Preview (First 5 rows):

   No.      Time      Source Destination Protocol  Length  \
0    1  0.000000    163.22.94.24   127.0.0.1      TCP      54
1    2  0.000017    36.199.134.38   127.0.0.1      TCP      54
2    3  0.000020   156.124.155.145   127.0.0.1      TCP      54
3    4  0.000023   219.253.152.220   127.0.0.1      TCP      54
4    5  0.000026    253.168.21.41   127.0.0.1      TCP      54

                                Info
0  2697  >  80 [SYN] Seq=0 Win=512 Len=0
1  2698  >  80 [SYN] Seq=0 Win=512 Len=0
2  2699  >  80 [SYN] Seq=0 Win=512 Len=0
3  2700  >  80 [SYN] Seq=0 Win=512 Len=0
4  2701  >  80 [SYN] Seq=0 Win=512 Len=0

Data Types (dtypes):
No.          int64
Time         float64
Source       object
Destination  object
Protocol     object
Length       int64
Info         object
dtype: object

Missing values (total): 0

Empty DataFrame
Columns: [missing_count]
Index: []

Key columns present: ['No.', 'Time', 'Source', 'Destination', 'Protocol', 'Length', 'Info']
Time range: 0.000000 ~ 4.735496 seconds
Length range: 54 ~ 54 bytes

```

3. 數據視覺化與診斷圖表分析

Wireshark 與 Python 分析的對比：在實驗中，Wireshark 提供了微觀的封包細節（如單個 TCP 標頭的 Seq/Ack 數值與單一封包長度），適合進行單點故障排除；而 Python 數據集與視覺化圖表則提供了宏觀的統計視角（如整體流量趨勢、協定比例與 IP 熵值）。兩者結合完整還原 DDoS 攻擊的全局樣貌。為避免僅以單一指標判定攻擊，我們建立四個分析維度，從流量總量、時間密度、結構特徵與來源分佈四個角度交叉驗證 SYN 洪泛攻擊。

3.1 Volume 維度：流量總量與成長斜率

如 Fig 1，在正常環境下，累積封包數隨時間呈現階梯式緩慢上升，反映一般瀏覽行為的間歇性請求特性。封包產生與頁面載入節奏相對應，並未出現持續性爆發。

相較之下 Fig 2 攻擊環境中累積封包數於 5 秒內近乎線性暴增至數百萬筆。其增長斜率遠高於正常環境，顯示封包生成速率已超出人類行為可能範圍。

Fig 1: Normal Environment - Cumulative Packets Trend

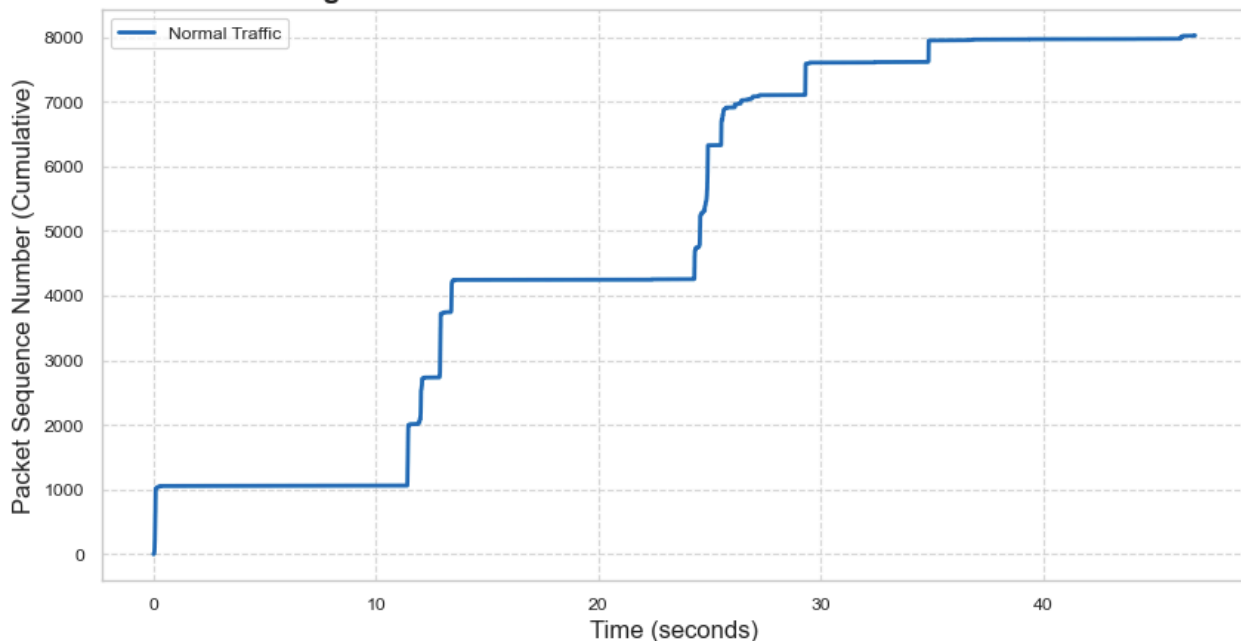
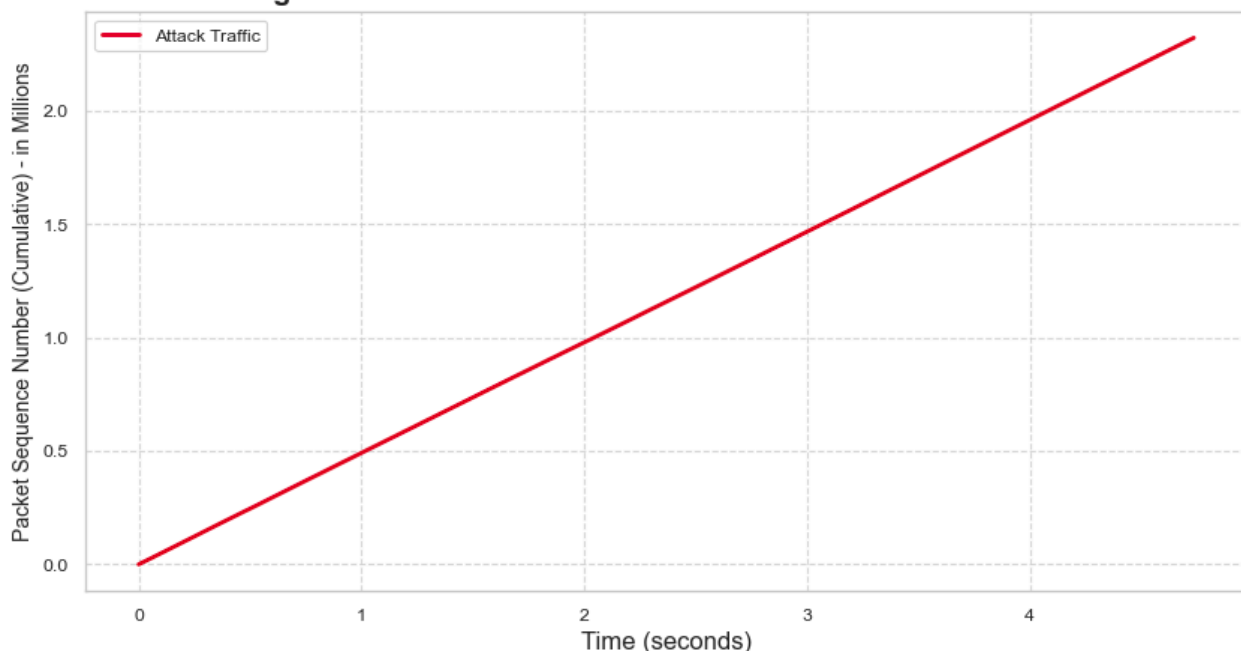


Fig 2: DDoS Attack Environment - Cumulative Packets Trend



3.2 Intensity 維度：時間密度分析

在時間密度 (Intensity) 的分析上，正常環境下的每秒封包數 (PPS) 維持在極低的穩定狀態，反映了常規網路請求的自然頻率。然而，在遭受 SYN Flood 攻擊期間，PPS 瞬間飆升，峰值高達數十萬 PPS。同時，觀察封包間隔時間 (IAT)，正常流量的分佈廣泛且具備自然波動，但攻擊流量的 IAT 卻高度集中於極小值。這種時間密度的異常 (Temporal Density Anomaly) 在數學上徹底違背了人類操作的隨機性，是自動化攻擊腳本的核心特徵。

Fig 8: Network Throughput Analysis (Packets Per Second)

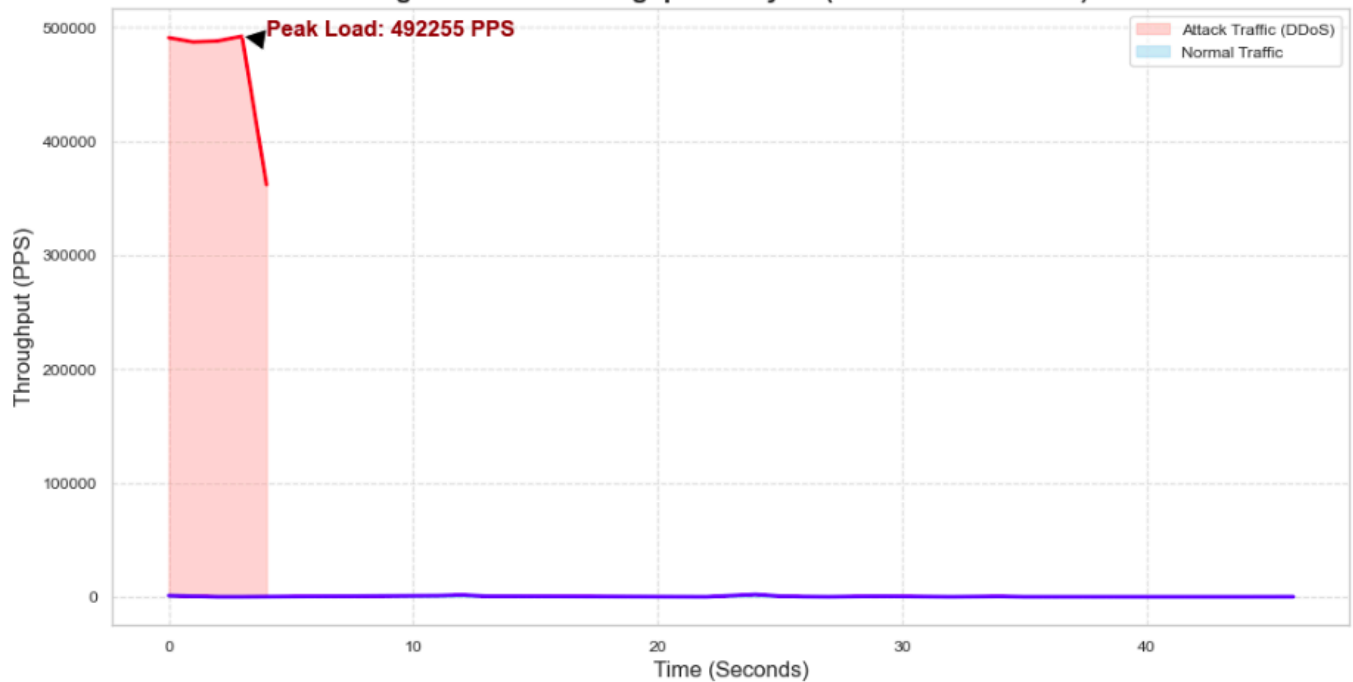
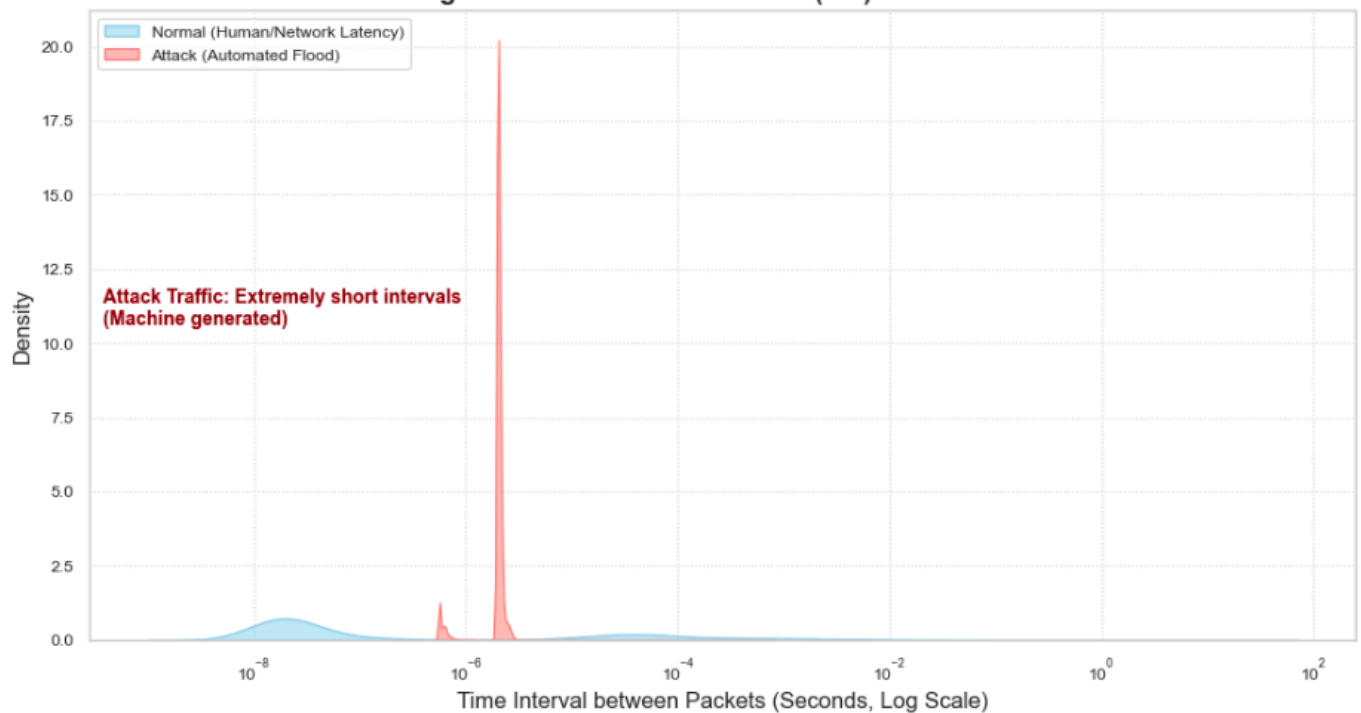


Fig 9: Packet Inter-Arrival Time (IAT) Distribution



3.3 Structure 維度: 協議與封包特徵

如 Fig 3(正常環境 TCP 柱狀圖)所示, 正常流量呈現高度的協定多樣性, 在不同時間區間內分散著 **QUIC**、**TLS**、**DNS** 與 **TCP** 等多種應用層與傳輸層協定。相對地, 對比 Fig 4(攻擊環境 TCP 柱狀圖)與進階協定分析, 攻擊期間的每一個 0.5 秒區間內都充滿了巨量的 TCP 封包, TCP 佔比高達 100%, 其他協定幾乎完全消失。這形成了一種極端的「協定單一化 (Protocol Monoculture)」」。此外, 攻擊流量的封包長度高度集中於 54 bytes (屬於無有效負載的 Header-only SYN 封包), 變異數為零, 進一步證實了流量的機械化特徵。

Fig 3: Normal Environment - TCP-like vs QUIC Packets (1s Interval)

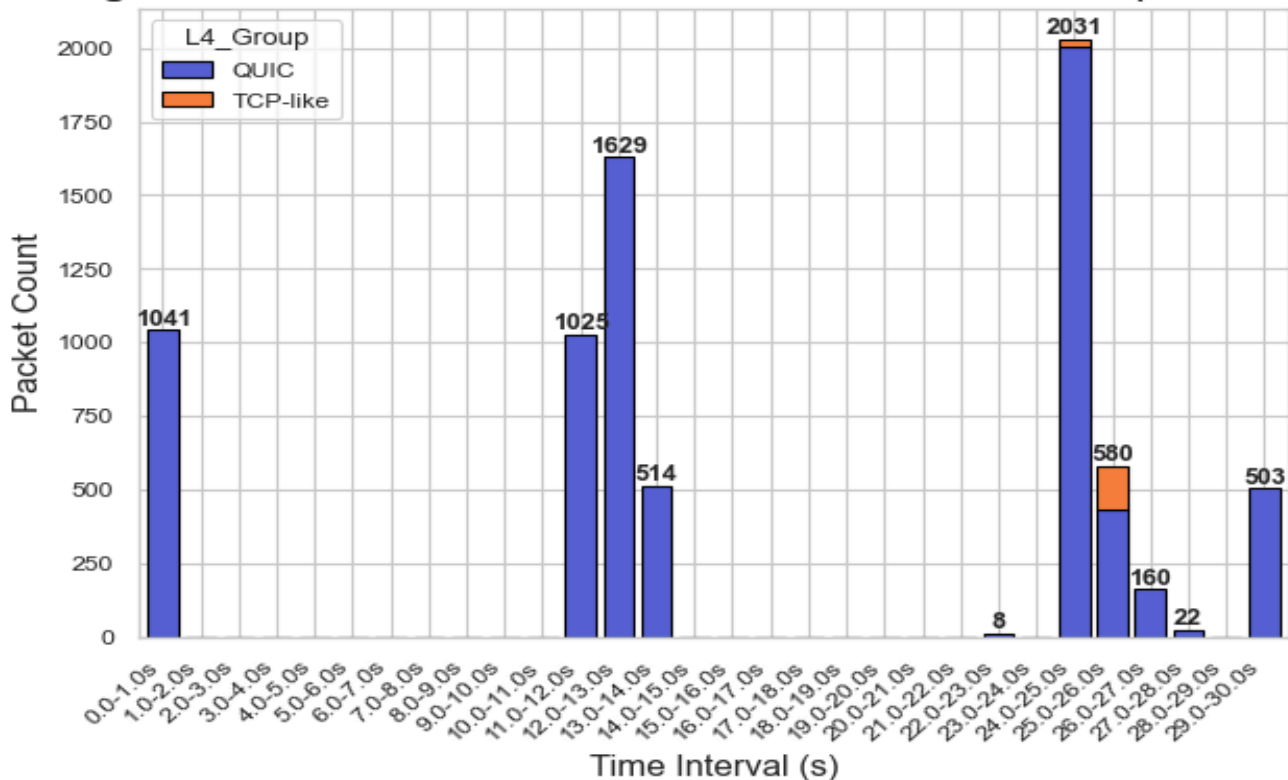
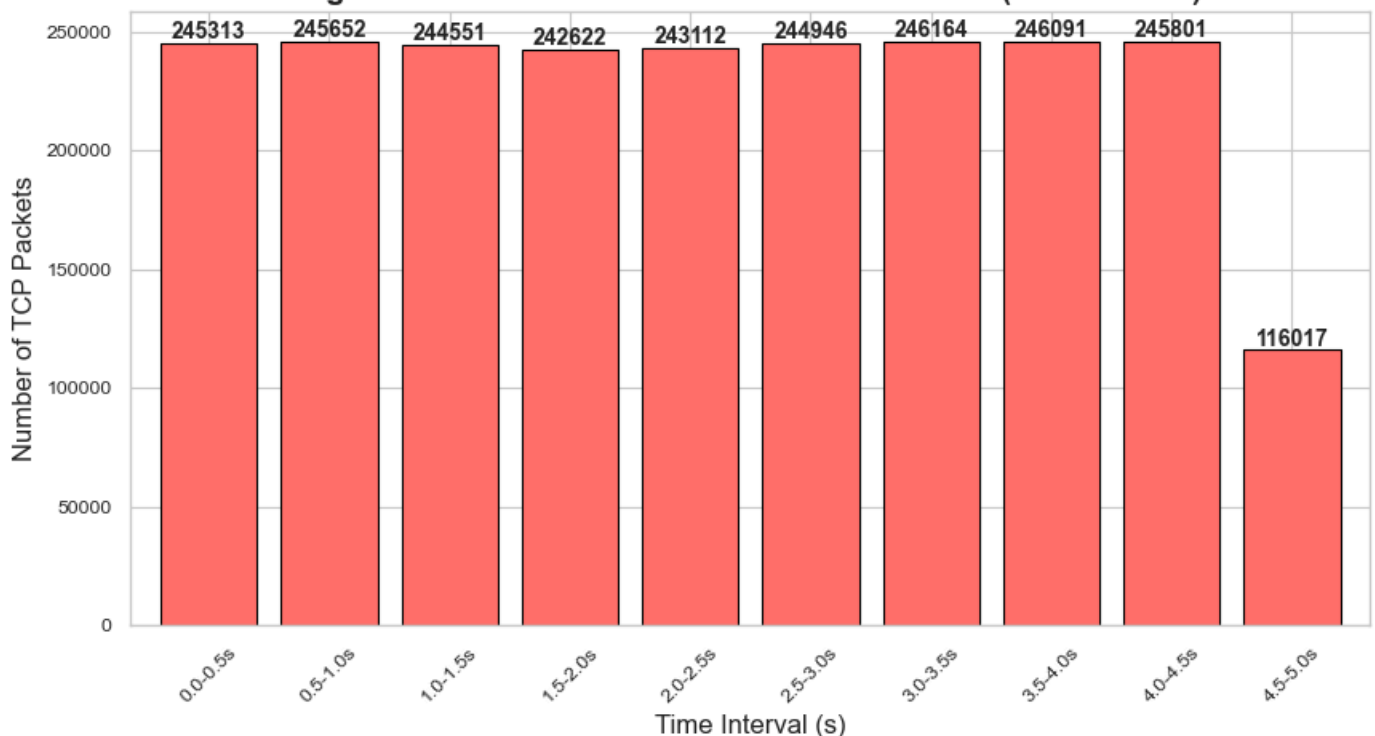
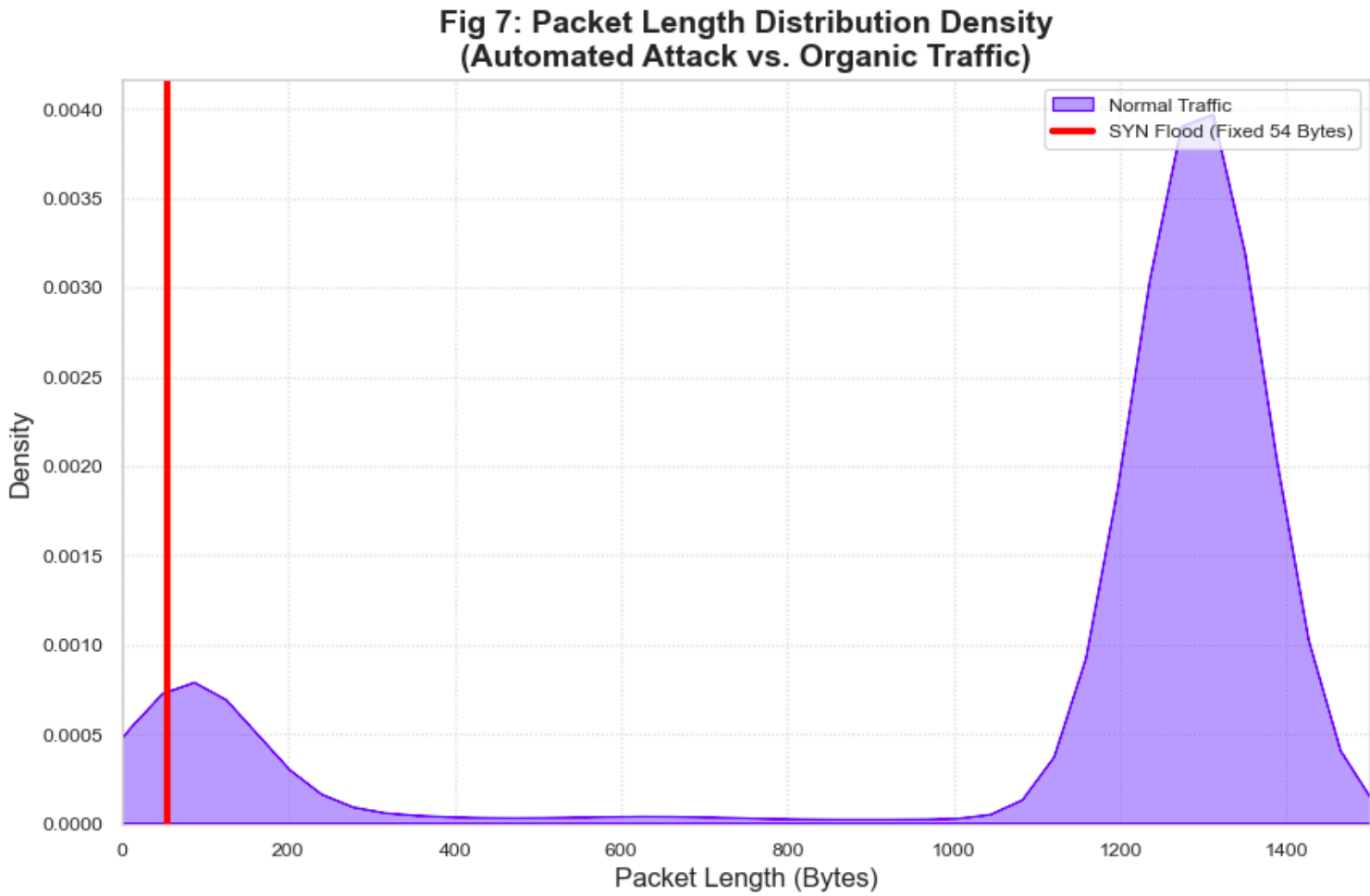
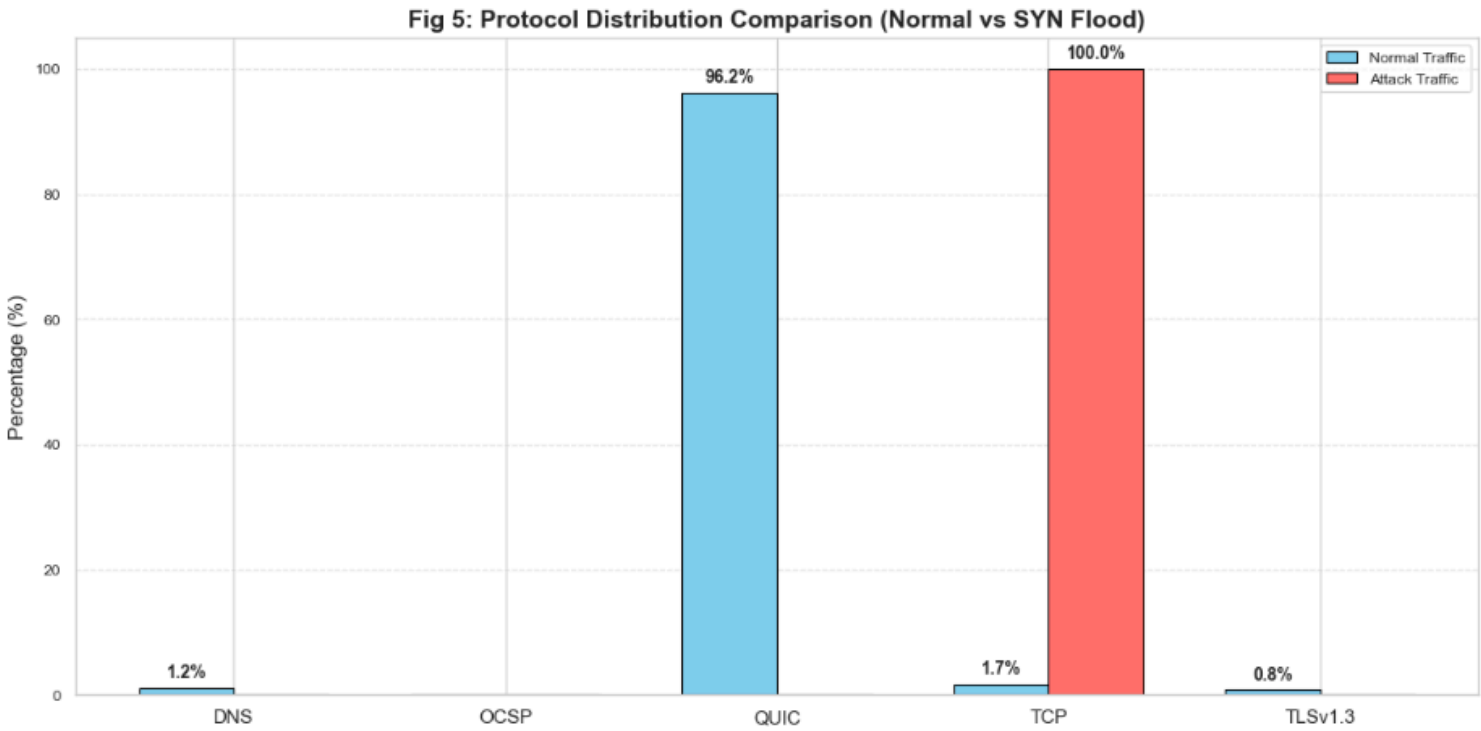


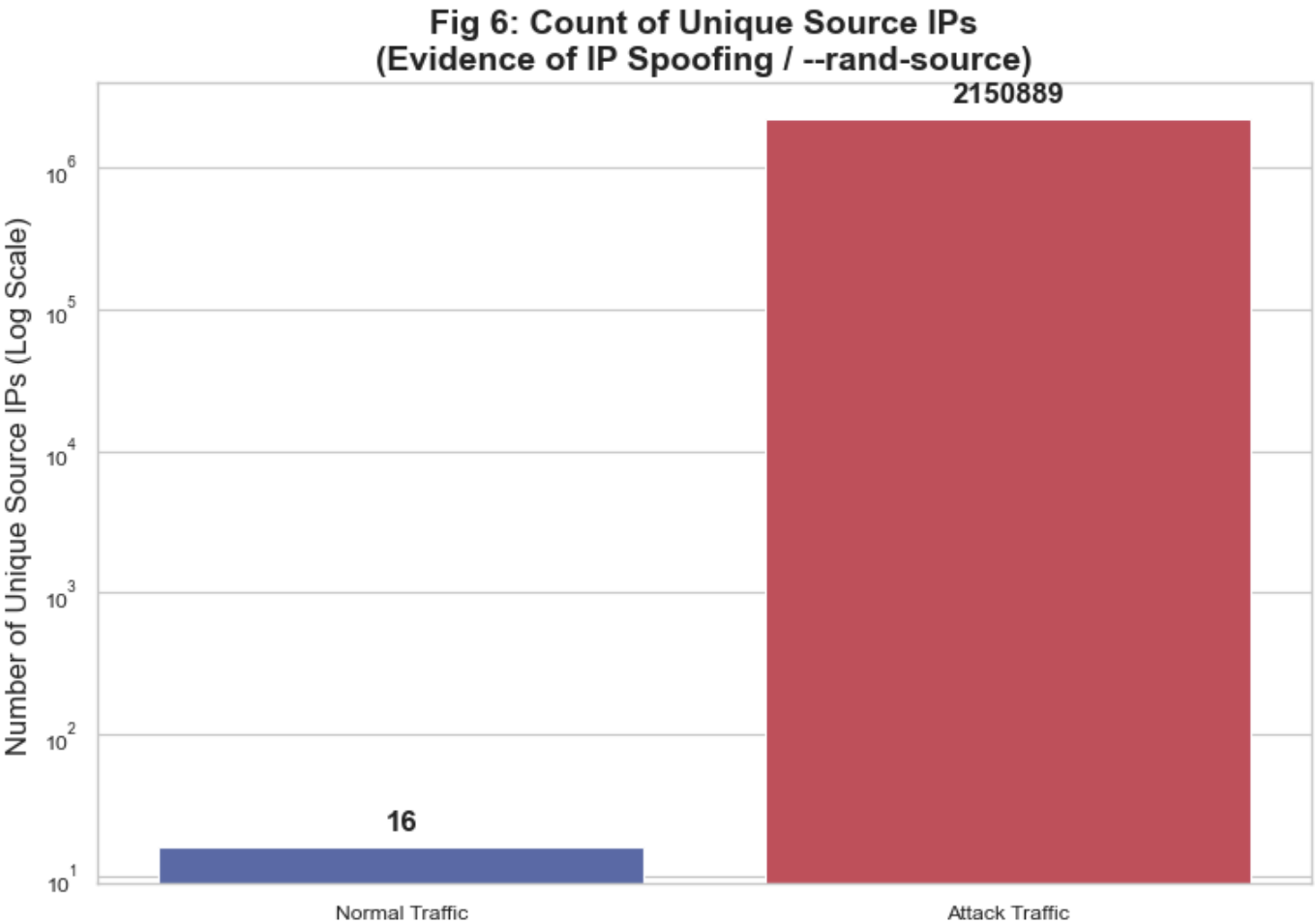
Fig 4: DDoS Attack Environment - TCP Packets (0.5s Interval)





3.4 Source Diversity 維度：來源分佈異常

在來源 IP 的多樣性分析中，正常流量僅產生極少數的來源 IP（主要為本地閘道與目標伺服器）。然而，攻擊流量的來源 IP 數量卻異常暴增至逾兩百萬個。這種來源多樣性異常升高 (Source Explosion) 的現象，直接證實了 `hping3` 指令中 `--rand-source` 參數所產生的 IP 欺騙 (IP Spoofing) 效果。真實用戶行為絕不可能在數秒內產生數百萬個不同的來源地址，此技術使得傳統基於單一 IP 黑名單的防火牆防禦完全失效。



3.5 綜合判定模型

維度	正常流量	攻擊流量
Volume	緩慢成長	爆炸性成長
Intensity	低 PPS / 高 IAT	高 PPS / 低 IAT
Structure	協議多樣	協議單一
Source	來源有限	來源爆炸

4. 結果分析與深度診斷

4.1 流量特徵與系統影響評估 (So What?)

攻擊流量並非單純的帶寬壓榨，而是一種**狀態耗盡 (State Exhaustion)**策略。

1. 網絡帶寬擁塞：雖然單個 SYN 包很小，但 `--flood` 產生的極高頻率會導致帶寬迅速飽和。
2. 中斷風暴 (Interrupt Storms)：海量 PPS 衝擊下，操作系統將花費幾乎所有 CPU 時間在「環境切換」與「中斷處理」上，導致正常應用程序無響應。
3. 吞吐量崩潰：當 CPU 和帶寬同時飽和，系統的有效吞吐量 (Goodput) 將趨近於零。

4.2 SYN 洪泛與「倉庫工人」類比

使用「倉庫工人」來類比 TCP 三向交握：

- 正常情況：工人 (伺服器) 收到訂單 (SYN)，準備好包裹 (SYN-ACK)，顧客確認 (ACK)，工人送出包裹。
- 攻擊情況：攻擊者不斷發送訂單 (SYN)，但不留下正確回信地址 (`--rand-source`)。工人將包裹堆在櫃檯 (Backlog Queue)，等待永遠不會來的確認。最終，櫃檯被填滿，新來的真正顧客被拒絕服務。

4.3 技術原理深挖：Backlog 與 tcpcb

從內核角度看，伺服器收到 SYN 時，會為該連線分配一個 TCP 控制塊 (tcpcb, TCP Control Block)。由於攻擊者不發送最後的 ACK，連線會長時間處於 `SYN_RECV` 狀態。

- 操作系統消耗大量內存維護「半開放連接」(Half-open connection) 隊列。
- 當隊列空間耗盡，內核會拒絕新的 SYN 請求。即使啟用了 `syncookies`，極高頻率的雜湊運算也會對 CPU 造成巨大負載。

4.4 程式語言與防禦算法的適用性

本實驗使用 Python 進行分析，因其具備數據科學生態系統 (Pandas, Matplotlib)。但在高性能防禦場景中，不同語言有其戰略職能：

- **C++**：常被用於入侵檢測系統 (IDS) 的底層開發，因其具備精確的內存控制和極低延遲。
- **Go 語言**：因其內建的高併發處理能力 (Goroutines)，適合開發高負載的網絡轉發組件或負載均衡器。

5. 進階研究: 非 SYN 洪泛類型的 DDoS 攻擊

除協議層攻擊外, 流量型 (Volumetric) 攻擊同樣具備毀滅性。

5.1 DNS 放大攻擊 (DNS Amplification) 機制

這是一種利用「放大倍率」的流量型攻擊。攻擊者向開放的 DNS 解析器發送偽造源地址 (受害者 IP) 的查詢請求。

- 放大原理: 一個 60 字節的請求可能引發 3000 字節以上的回應。
- 對比: 與 SYN 洪泛旨在耗盡「伺服器連接狀態」不同, DNS 放大攻擊旨在「塞滿受害者的帶寬水管」(Layer 3/4 頻寬壓榨)。

5.2 真實案例分析: GitHub 遭受的 Memcached 放大攻擊

2018 年 GitHub 遭受了峰值高達 1.35 Tbps 的 DDoS 攻擊, 利用 Memcached 服務器的不當配置, 將流量放大了驚人的 50,000 倍 (NIST, 2023)。這證明了單點防禦在面對大規模放大攻擊時的無效。

5.3 應對措施評估

針對此類攻擊, 必須採取多層次策略:

1. 流量清洗 (**Scrubbing**): 將流量引導至專業供應商 (如 Cloudflare), 利用大規模分佈式節點識別並丟棄惡意包。
2. 任播路由 (**Anycast**): 將攻擊流量分散到全球多個數據中心, 避免單一入口崩潰。
3. **WAF** 與行為分析: 在應用層識別異常的 HTTP 請求特徵, 在連線完全建立前進行攔截。

6. 結論與建議

本報告透過實證與數據分析流程，成功揭示了 SYN 洪泛攻擊如何利用 TCP 三向交握的設計缺陷，以極低成本耗盡目標伺服器的系統資源。我們透過 Wireshark 的微觀捕獲與 Python 的宏觀診斷性分析，我們明確識別出攻擊流量的四大核心指紋：垂直飆升的封包生成速率（高 PPS）、極端單一的 TCP 協定佔比、高度一致的封包長度（固定 54 bytes），以及 `--rand-source` 所引發的來源 IP 異常暴增。

這些特徵在正常的人類瀏覽行為中是絕不可能出現的，充分證明了自動化攻擊腳本的機械化本質。

針對此類與日俱增的網路威脅，單純依賴擴充網路頻寬已無法解決所有 DDoS 問題（如應用層攻擊或狀態耗盡攻擊）。未來的資安防禦必須高度依賴數據分析技術，建議組織採取以下防禦思維：

1. 建立動態流量基線：利用機器學習或數據可視化工具，持續監控日常 PPS、協定比例與連線狀態，一旦偏離基線即觸發警報。
2. 落實協定層防護：針對 SYN Flood，應於系統核心啟用 SYN Cookies，並合理設定 Backlog 佇列大小，避免半開連線癱瘓服務。
3. 多層次架構抗壓：結合 ISP 源頭過濾（防範 IP 欺騙）、Anycast 流量分散與專業的雲端清洗中心（Scrubbing Center），在惡意封包抵達伺服器前進行攔截。

總結而言，網路安全已從「規則阻擋」演進為「數據驅動」的對抗。唯有具備深度的封包解析能力與自動化的數據洞察，組織才能在複雜的網路攻擊中實現快速偵測與精準阻斷。

7. 參考文獻

- Course Materials. Unit 2: Basic Data Analysis with Python. UTS 421107 Cybersecurity Analytics and Insights. <https://canvas.uts.edu.au>
- Course Materials. Unit 3: Packet Analysis & DDoS Attacks. UTS 421107 Cybersecurity Analytics and Insights. <https://canvas.uts.edu.au>
- Cloudflare. (n.d.). What is a SYN flood attack?
<https://www.cloudflare.com/learning/ddos/ddos-attack-types/syn-flood/>
- NIST. (2023). Guide to Information Technology Security Services.
<https://csrc.nist.gov/publications/detail/sp/800-100/final>
- Cisco. (2023). Defending Against Distributed Denial of Service (DDoS) Attacks.
<https://www.cisco.com/c/en/us/support/docs/security/firepower-ngfw/214341-ddos-defense-guide.html>